

Experiment 'p_4_1_c_powers_of_two_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_1_c_powers_of_two_seq

Natural language statement: null

Method signature: p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

Ensures

- |powers| == 21
- forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_1_c_powers_of_two_seq()` returns `(powers: seq<int>)`

The method should respect the following contract:

`ensures |powers| == 21, ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)`

The contract uses the following dafny code:

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLAERjjWvF3k97wf2ksBgsq8Ff6P

Model: gpt-5.5-2026-04-23

Created at: 1780651798

Finish reason: stop

Usage: 438 tokens in, and 1176 tokens out

```
{
  powers := [
    1,
    2,
    4,
    8,
    16,
    32,
    64,
    128,
```

```

    256,
    512,
    1024,
    2048,
    4096,
    8192,
    16384,
    32768,
    65536,
    131072,
    262144,
    524288,
    1048576
  ];
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(11,0): Error: a postcondition could
  ↳ not be proved on this return path

```

```

|
11 | {
|   ^

```

```

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(10,9): Related location: this is the
  ↳ postcondition that could not be proved

```

```

|
10 | ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
|   ~~~~~~

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLB1aoKKGOvzbEXZNpGGZYEqOMj8

Model: gpt-5.5-2026-04-23

Created at: 1780651847

Finish reason: stop

Usage: 784 tokens in, and 3806 tokens out

```

{
  powers := [
    1,

```

```

2,
4,
8,
16,
32,
64,
128,
256,
512,
1024,
2048,
4096,
8192,
16384,
32768,
65536,
131072,
262144,
524288,
1048576
];

assert |powers| == 21;

assert pow(2, 0) == 1;
assert pow(2, 1) == 2 * pow(2, 0);
assert pow(2, 1) == 2;
assert pow(2, 2) == 2 * pow(2, 1);
assert pow(2, 2) == 4;
assert pow(2, 3) == 2 * pow(2, 2);
assert pow(2, 3) == 8;
assert pow(2, 4) == 2 * pow(2, 3);
assert pow(2, 4) == 16;
assert pow(2, 5) == 2 * pow(2, 4);
assert pow(2, 5) == 32;
assert pow(2, 6) == 2 * pow(2, 5);
assert pow(2, 6) == 64;
assert pow(2, 7) == 2 * pow(2, 6);
assert pow(2, 7) == 128;
assert pow(2, 8) == 2 * pow(2, 7);
assert pow(2, 8) == 256;
assert pow(2, 9) == 2 * pow(2, 8);
assert pow(2, 9) == 512;
assert pow(2, 10) == 2 * pow(2, 9);
assert pow(2, 10) == 1024;
assert pow(2, 11) == 2 * pow(2, 10);
assert pow(2, 11) == 2048;
assert pow(2, 12) == 2 * pow(2, 11);
assert pow(2, 12) == 4096;
assert pow(2, 13) == 2 * pow(2, 12);
assert pow(2, 13) == 8192;

```

```

assert pow(2, 14) == 2 * pow(2, 13);
assert pow(2, 14) == 16384;
assert pow(2, 15) == 2 * pow(2, 14);
assert pow(2, 15) == 32768;
assert pow(2, 16) == 2 * pow(2, 15);
assert pow(2, 16) == 65536;
assert pow(2, 17) == 2 * pow(2, 16);
assert pow(2, 17) == 131072;
assert pow(2, 18) == 2 * pow(2, 17);
assert pow(2, 18) == 262144;
assert pow(2, 19) == 2 * pow(2, 18);
assert pow(2, 19) == 524288;
assert pow(2, 20) == 2 * pow(2, 19);
assert pow(2, 20) == 1048576;

forall i | 0 <= i < |powers|
  ensures powers[i] == pow(2, i)
{
  if i == 0 {
    assert powers[i] == 1;
    assert pow(2, i) == 1;
  } else if i == 1 {
    assert powers[i] == 2;
    assert pow(2, i) == 2;
  } else if i == 2 {
    assert powers[i] == 4;
    assert pow(2, i) == 4;
  } else if i == 3 {
    assert powers[i] == 8;
    assert pow(2, i) == 8;
  } else if i == 4 {
    assert powers[i] == 16;
    assert pow(2, i) == 16;
  } else if i == 5 {
    assert powers[i] == 32;
    assert pow(2, i) == 32;
  } else if i == 6 {
    assert powers[i] == 64;
    assert pow(2, i) == 64;
  } else if i == 7 {
    assert powers[i] == 128;
    assert pow(2, i) == 128;
  } else if i == 8 {
    assert powers[i] == 256;
    assert pow(2, i) == 256;
  } else if i == 9 {
    assert powers[i] == 512;
    assert pow(2, i) == 512;
  } else if i == 10 {
    assert powers[i] == 1024;
    assert pow(2, i) == 1024;
  }
}

```

```

    } else if i == 11 {
      assert powers[i] == 2048;
      assert pow(2, i) == 2048;
    } else if i == 12 {
      assert powers[i] == 4096;
      assert pow(2, i) == 4096;
    } else if i == 13 {
      assert powers[i] == 8192;
      assert pow(2, i) == 8192;
    } else if i == 14 {
      assert powers[i] == 16384;
      assert pow(2, i) == 16384;
    } else if i == 15 {
      assert powers[i] == 32768;
      assert pow(2, i) == 32768;
    } else if i == 16 {
      assert powers[i] == 65536;
      assert pow(2, i) == 65536;
    } else if i == 17 {
      assert powers[i] == 131072;
      assert pow(2, i) == 131072;
    } else if i == 18 {
      assert powers[i] == 262144;
      assert pow(2, i) == 262144;
    } else if i == 19 {
      assert powers[i] == 524288;
      assert pow(2, i) == 524288;
    } else {
      assert i == 20;
      assert powers[i] == 1048576;
      assert pow(2, i) == 1048576;
    }
  }
}

```

Final Program

```

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)
  ensures |powers| == 21
  ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
{
  powers := [
    1,

```

```

2,
4,
8,
16,
32,
64,
128,
256,
512,
1024,
2048,
4096,
8192,
16384,
32768,
65536,
131072,
262144,
524288,
1048576
];

assert |powers| == 21;

assert pow(2, 0) == 1;
assert pow(2, 1) == 2 * pow(2, 0);
assert pow(2, 1) == 2;
assert pow(2, 2) == 2 * pow(2, 1);
assert pow(2, 2) == 4;
assert pow(2, 3) == 2 * pow(2, 2);
assert pow(2, 3) == 8;
assert pow(2, 4) == 2 * pow(2, 3);
assert pow(2, 4) == 16;
assert pow(2, 5) == 2 * pow(2, 4);
assert pow(2, 5) == 32;
assert pow(2, 6) == 2 * pow(2, 5);
assert pow(2, 6) == 64;
assert pow(2, 7) == 2 * pow(2, 6);
assert pow(2, 7) == 128;
assert pow(2, 8) == 2 * pow(2, 7);
assert pow(2, 8) == 256;
assert pow(2, 9) == 2 * pow(2, 8);
assert pow(2, 9) == 512;
assert pow(2, 10) == 2 * pow(2, 9);
assert pow(2, 10) == 1024;
assert pow(2, 11) == 2 * pow(2, 10);
assert pow(2, 11) == 2048;
assert pow(2, 12) == 2 * pow(2, 11);
assert pow(2, 12) == 4096;
assert pow(2, 13) == 2 * pow(2, 12);
assert pow(2, 13) == 8192;

```

```

assert pow(2, 14) == 2 * pow(2, 13);
assert pow(2, 14) == 16384;
assert pow(2, 15) == 2 * pow(2, 14);
assert pow(2, 15) == 32768;
assert pow(2, 16) == 2 * pow(2, 15);
assert pow(2, 16) == 65536;
assert pow(2, 17) == 2 * pow(2, 16);
assert pow(2, 17) == 131072;
assert pow(2, 18) == 2 * pow(2, 17);
assert pow(2, 18) == 262144;
assert pow(2, 19) == 2 * pow(2, 18);
assert pow(2, 19) == 524288;
assert pow(2, 20) == 2 * pow(2, 19);
assert pow(2, 20) == 1048576;

forall i | 0 <= i < |powers|
  ensures powers[i] == pow(2, i)
{
  if i == 0 {
    assert powers[i] == 1;
    assert pow(2, i) == 1;
  } else if i == 1 {
    assert powers[i] == 2;
    assert pow(2, i) == 2;
  } else if i == 2 {
    assert powers[i] == 4;
    assert pow(2, i) == 4;
  } else if i == 3 {
    assert powers[i] == 8;
    assert pow(2, i) == 8;
  } else if i == 4 {
    assert powers[i] == 16;
    assert pow(2, i) == 16;
  } else if i == 5 {
    assert powers[i] == 32;
    assert pow(2, i) == 32;
  } else if i == 6 {
    assert powers[i] == 64;
    assert pow(2, i) == 64;
  } else if i == 7 {
    assert powers[i] == 128;
    assert pow(2, i) == 128;
  } else if i == 8 {
    assert powers[i] == 256;
    assert pow(2, i) == 256;
  } else if i == 9 {
    assert powers[i] == 512;
    assert pow(2, i) == 512;
  } else if i == 10 {
    assert powers[i] == 1024;
    assert pow(2, i) == 1024;
  }
}

```

```
    } else if i == 11 {
        assert powers[i] == 2048;
        assert pow(2, i) == 2048;
    } else if i == 12 {
        assert powers[i] == 4096;
        assert pow(2, i) == 4096;
    } else if i == 13 {
        assert powers[i] == 8192;
        assert pow(2, i) == 8192;
    } else if i == 14 {
        assert powers[i] == 16384;
        assert pow(2, i) == 16384;
    } else if i == 15 {
        assert powers[i] == 32768;
        assert pow(2, i) == 32768;
    } else if i == 16 {
        assert powers[i] == 65536;
        assert pow(2, i) == 65536;
    } else if i == 17 {
        assert powers[i] == 131072;
        assert pow(2, i) == 131072;
    } else if i == 18 {
        assert powers[i] == 262144;
        assert pow(2, i) == 262144;
    } else if i == 19 {
        assert powers[i] == 524288;
        assert pow(2, i) == 524288;
    } else {
        assert i == 20;
        assert powers[i] == 1048576;
        assert pow(2, i) == 1048576;
    }
}
}
```

Total Token Usage

Input tokens: 1222

Output tokens: 4982

Reasoning tokens: 3169

Sum of 'total tokens': 6204

Experiment Timings

Overall Experiment started at 1780651797454, ended at 1780651930632, lasting 133178ms (133.18 seconds)

Iteration #1 started at 1780651797460, ended at 1780651846772, lasting 49312ms (49.31 seconds)

Iteration #2 started at 1780651846781, ended at 1780651930632, lasting 83851ms (83.85 seconds)